DX Audit Report

Version 1.0

Published by Dionis Xhaferi

5/11/25

Summary

This report provides an in-depth security assessment of your Solana Anchor-based presale contract. The contract enables token distribution in exchange for USDC with tiered pricing based on sales milestones. It enforces purchase limits per wallet and a hard cap to control total token distribution. The contract also includes functionality to start and end the presale, controlled by the contract owner. While the contract achieves its functional goals, several critical and medium-severity issues were identified that could lead to economic imprecision, misuse of token accounts, or reduced auditability. This report outlines these issues and provides actionable code-level recommendations to mitigate them.

Findings Overview

ID	Severity	Title
[H-1]	High	Price Calculation Precision Loss
[M-1]	Medium	Missing Validation of Token Account Ownership
[M-2]	Medium	Reentrancy-Like Issue Due to Unchecked State Updates
[L-1]	Low	Missing Event Emissions for Transparency
[I-1]	Informational	Unchecked Unix Timestamp Handling

Detailed Findings

[H-1] Price Calculation Precision Loss

Description

The contract currently uses floating-point (`f64`) arithmetic to compute tiered prices for token purchases. Floating-point calculations on blockchain platforms are inherently risky due to rounding and precision errors. Solana programs are expected to operate deterministically using integer math to ensure consistency across all validators. Relying on floating- point arithmetic introduces inconsistencies that could result in unfair pricing, overcharging, or undercharging buyers.

Impact

This vulnerability could cause users to be charged inaccurately, resulting in disputes or financial loss. Additionally, advanced users could exploit floating-point rounding to pay less than they should.

Proof of Concept

```
let tier_price = (state.initial_price as f64 * 1.28f64.powf(current_tier as f64)) as u64;
```

Recommended Mitigation

```
const TIER_MULTIPLIER_BASIS_POINTS: u64 = 12800; const
  BASIS_POINTS_DIVISOR: u64 = 10000;

let mut tier_price = state.initial_price; for _
  in 0..current_tier {
tier_price = tier_price
.checked_mul(TIER_MULTIPLIER_BASIS_POINTS)
.ok_or(PresaleError::MathOverflow)?
.checked_div(BASIS_POINTS_DIVISOR)
.ok_or(PresaleError::MathOverflow)?;
  }
```

[M-1] Missing Validation of Token Account Ownership

Description

The contract currently accepts any token account as the `buyer_usdc_account` without verifying that it belongs to the buyer. This could allow malicious actors to specify a token account they don't control, potentially draining third-party accounts.

Impact

This could lead to unauthorized token transfers from accounts the buyer does not own, which would violate user expectations and security best practices.

Proof of Concept

```
buyer_usdc_account.owner
```

Recommended Mitigation

```
#[account(
mut,
constraint = buyer_usdc_account.owner == buyer.key()
  )]
pub buyer_usdc_account: Account<'info, TokenAccount>,
```

[M-2] Reentrancy-Like Issue Due to Unchecked State Updates

Description

The contract updates its state only after transferring tokens. While Solana doesn't support reentrancy in the same way as EVM-based chains, it's best practice to update critical state before performing external actions to avoid inconsistencies in case of failure.

Impact

Leaving state updates until after external calls can open up subtle edge cases where state does not reflect the true status if the transaction fails halfway.

Proof of Concept

```
token::transfer(cpi_ctx, total_cost)?;
  user_purchase.amount += desired_amount; state.total_sold
  += desired_amount;
```

Recommended Mitigation

```
user_purchase.amount += desired_amount; state.total_sold
  += desired_amount;


  let cpi_ctx = CpiContext::new(
      ctx.accounts.token_program.to_account_info(),
      Transfer {
from: ctx.accounts.buyer_usdc_account.to_account_info(), to:
          ctx.accounts.treasury_account.to_account_info(), authority:
          ctx.accounts.buyer.to_account_info(),
      },
  );
token::transfer(cpi_ctx, total_cost)?;
```

[L-1] Missing Event Emissions for Transparency

Description

The contract does not emit any events after critical operations like token purchases. Events are essential for off-chain systems, such as block explorers, indexers, and monitoring tools, to track what happens on-chain.

Impact

Without events, it is difficult to audit or index presale activity, reducing transparency and making external integrations harder.

Recommended Mitigation

```
#[event]
  pub struct PurchaseEvent { pub
      buyer: Pubkey,
pub amount: u64, pub cost:
      u64,
  }

emit!(PurchaseEvent {
buyer: ctx.accounts.buyer.key(), amount:
      desired_amount,
cost: total_cost,
  });
```

[I-1] Unchecked Unix Timestamp Handling

Description

The contract records the current Unix timestamp without verifying its validity. While unlikely, a malicious validator could provide an incorrect timestamp.

Impact

This could impact any future logic that depends on the `start_time` value.

Recommended Mitigation

```
let current_time = Clock::get()?.unix_timestamp; require!(current_time >
  0, PresaleError::InvalidTimestamp); state.start_time = current_time;
```
